Journal Name

ARTICLE TYPE

Cite this: DOI: 00.0000/xxxxxxxxxx

Presenting MetaDesign — A Multi-Paradigm Active Learning Framework for the Inverse Design of Sustainable Cementitious Materials†

Ghezal Ahmad Jan Zia,*[a] Full Name,[b‡] and Full Name[a]

Received Date
Accepted Date

DOI: 00.0000/xxxxxxxxxx

The design of sustainable cementitious materials is challenged by growing formulation complexity, raw material variability, and the need to simultaneously optimize mechanical performance, carbon footprint, and cost. Sequential learning and inverse design (ID) approaches have demonstrated that materials discovery can be accelerated with substantially reduced experimental effort relative to conventional trial-and-error. Building upon the open-source SLAMD framework,[1] we introduce **MetaDesign** — a multi-paradigm active learning (AL) platform that substantially extends the modelling and exploration capabilities available to laboratory practitioners. MetaDesign integrates ten surrogate model architectures: Gaussian Process (GP), Random Forest (RF), calibrated Lolopy RF, Physics-Informed Neural Network (PINN), Deep Kernel Learning (DKL), Model-Agnostic Meta-Learning (MAML), Reptile, Prototypical Networks (ProtoNet), Reinforcement Learning via Proximal Policy Optimization (RL–PPO), and an uncertainty-weighted Ensemble. Three operating modes — pure machine learning (ML Mode), pure large language model agent reasoning (LLM Agent Mode), and a novel hybrid fusion mode (Hybrid Mode) — enable practitioners to leverage statistical inference and domain knowledge simultaneously. A unified WEBSLAMD-compatible utility function, four pluggable acquisition functions, multi-objective optimization, batch selection with diversity weighting, an interactive design space builder, and a comprehensive visualization dashboard are delivered through a web-based interface built on Flask, PyTorch, and scikit-learn. This paper describes the conceptual design of MetaDesign, its architecture and workflow, and demonstrates how it lowers the barrier to data-driven inverse design of sustainable concrete.

Concrete is the most extensively produced construction material worldwide, with an estimated annual output of approximately 30 billion tonnes.[1] The production of Portland cement clinker is energy-intensive and contributes roughly 8% of global CO_2 emissions, making the development of supplementary cementitious materials (SCMs), alkali-activated binders, and low-carbon admixtures a critical research priority.[2,3]

The formulation space for modern sustainable concrete is high-dimensional. Blended binders, activators, recycled aggregates, and chemical admixtures collectively create a combinatorial design space that cannot be systematically explored by trial and error within realistic laboratory budgets.[1] Inverse design (ID) methods address this by coupling predictive surrogate models with acquisition functions that intelligently select the most informative experiments in an iterative, adaptive loop.[4,5]

The SLAMD framework[1] operationalized sequential learning (SL) for building materials in an accessible web application, showing that competitive alkali-activated binder formulations can be identified with up to 60 times less data than conventional machine learning approaches.[2] Despite these advances, the surrogate model choices available in existing tools remain narrow, exploration is purely statistical, and emerging paradigms — including large language models (LLMs), meta-learning, physics-informed neural networks (PINNs), and reinforcement learning (RL) — remain largely unexploited in materials discovery software.

Here we introduce **MetaDesign**, an open-source multi-paradigm AL platform that integrates ten surrogate models, three operating modes including a novel LLM–ML hybrid, four acquisition functions, and a suite of visualization tools into a unified web application compatible with the WEBSLAMD utility framework.[1]

[a] *Address, Address, Town, Country. Fax: XX XXXX XXXX; Tel: XX XXXX XXXX; E-mail: xxxx@aaa.bbb.ccc*

[b] *Address, Address, Town, Country.*

† Additional footnotes to the title and authors can be included *e.g.* 'Present address:' or 'These authors contributed equally to this work' as above using the symbols: ‡, §, and ¶. Please place the appropriate symbol next to the author's name and include a `\footnotetext` entry in the the correct place in the list.

1 Background and related work

1.1 Sequential learning for materials

Sequential learning couples a surrogate model with an acquisition function in an iterative Bayesian experimental design loop. Rohr *et al.* demonstrated up to 20-fold acceleration in materials discovery over random selection.[6] Ling *et al.* reported three-fold improvements in finding optimal solutions across multiple materials domains.[7] Völker *et al.* applied this paradigm to alkali-activated binders, requiring up to 60 times less data than conventional ML[2] and subsequently showed that a-priori information such as CO_2 footprint can be incorporated without sacrificing mechanical performance.[3]

1.2 Physics-informed and meta-learning models

Physics-informed neural networks (PINNs)[8] encode domain laws directly into the training loss, enabling physically consistent extrapolation in data-sparse regimes. Model-agnostic meta-learning (MAML)[9] and its first-order variants (Reptile[10]) learn parameter initializations from which models can rapidly adapt to new tasks with minimal data — a natural fit for materials campaigns where new raw material sources frequently arise. Prototypical Networks[11] provide distance-based few-shot inference. Deep Kernel Learning (DKL) combines neural network feature extraction with Gaussian Process uncertainty, inheriting the calibrated probabilistic output of the GP while overcoming its fixed-kernel limitation.

1.3 LLM-assisted optimization

Large language models encode extensive domain knowledge from scientific literature and can reason about materials properties in natural language. Recent work has demonstrated LLM-guided experimental design in chemistry,[12] but integration with statistical AL pipelines for construction materials has not previously been reported. MetaDesign's Hybrid Mode provides the first such integration for cementitious materials design.

1.4 Reinforcement learning for experiment selection

Reinforcement learning agents learn adaptive selection policies through trial and error, potentially discovering system-specific exploration strategies that outperform fixed acquisition functions over extended campaigns.[13] Prior RL applications in materials science have focused on molecular design[14] rather than experiment selection in physical laboratories, leaving this paradigm unexplored in the building materials domain.

2 The inverse design method

MetaDesign adopts the ID paradigm as formalized in Völker *et al.*[1,2] and Zunger.[4] The design space (DS) is a high-dimensional vector space in which each material formulation is represented by its composition and processing parameters. The DS can be constructed using MetaDesign's Design Space Builder or uploaded as a CSV/Excel file.

The WEBSLAMD utility function[1] governs candidate scoring across all surrogate models:

$$u_i = \sum_j \left[w_j \cdot \frac{\hat{y}_{ij} - \mu_j}{\sigma_j} \cdot s_j \right] + \alpha \sum_j \left[w_j \cdot \frac{\hat{\sigma}_{ij}}{\sigma_j} \right] \tag{1}$$

where μ_j and σ_j are the mean and standard deviation of labeled target values (updated per iteration), $s_j \in \{+1, -1\}$ encodes the optimization direction (maximize or minimize), w_j are user-assigned importance weights, $\hat{\sigma}_{ij}$ is the predicted uncertainty, and $\alpha \in [0,1]$ is the curiosity parameter controlling the exploration–exploitation balance. At $\alpha = 0$ the engine is purely exploitative; at $\alpha = 1$ it prioritizes the most uncertain candidates.

An additional novelty score $\mathcal{N}_i$ — the normalized mean distance to the k=5 nearest labeled samples in feature space — provides an optional bonus term in the Bayesian optimizer: $u_i \mathrel{+}= \beta \cdot \alpha \cdot \mathcal{N}_i$, where β is the novelty weight.

The complete framework is illustrated in Fig. 1. The design space feeds into three parallel operating modes that share a common bottom-row pipeline: diversity-aware batch selection, ranked candidate output with uncertainty estimates and t-SNE visualization, laboratory testing, and dataset update for the next AL cycle. The curiosity parameter α (Eq. 1) is user-controlled across all modes via the explore–exploit dial shown in the figure. Typical inputs include water-to-binder ratio, SCM content, aggregate fraction, and admixture dosage; typical targets include compressive strength (maximize), CO_2 footprint (minimize), and workability (maximize).

3 MetaDesign: architecture and features

MetaDesign is implemented as a Flask web application with a Python backend and a Vanilla JavaScript frontend. PyTorch provides neural network model training, scikit-learn provides classical ML models, and SQLite via SQLAlchemy handles project and scenario persistence. Response compression (`flask-compress`) and rate limiting (`flask-limiter`, 200 requests/day per IP) are applied for production robustness. The landing page and main navigation are shown in Fig. 2.

3.1 Surrogate model library

All ten surrogate models expose a unified `predict_with_uncertainty(X)` interface returning predictions, uncertainty estimates, and optionally posterior samples, ensuring that all acquisition functions and utility computations remain model-agnostic. Table 1 summarizes the portfolio.

3.1.1 Gaussian Process.

The GP model uses a Matérn kernel combined with a WhiteKernel for noise modelling, fitting a separate GP per target column. With `n_restarts_optimizer = 10` for robust hyperparameter optimization,[15] the GP provides closed-form posterior uncertainty and is the most theoretically grounded choice for small datasets ($n < 500$), though its $O(n^3)$ complexity limits scalability.

3.1.2 Random Forest models.

The RF model wraps scikit-learn's `RandomForestRegressor` (100 trees, StandardScaler) and estimates uncertainty from inter-tree variance. The Lolopy RF variant[16] uses jackknife-after-bootstrap uncertainty, producing calibrated prediction intervals where 68%

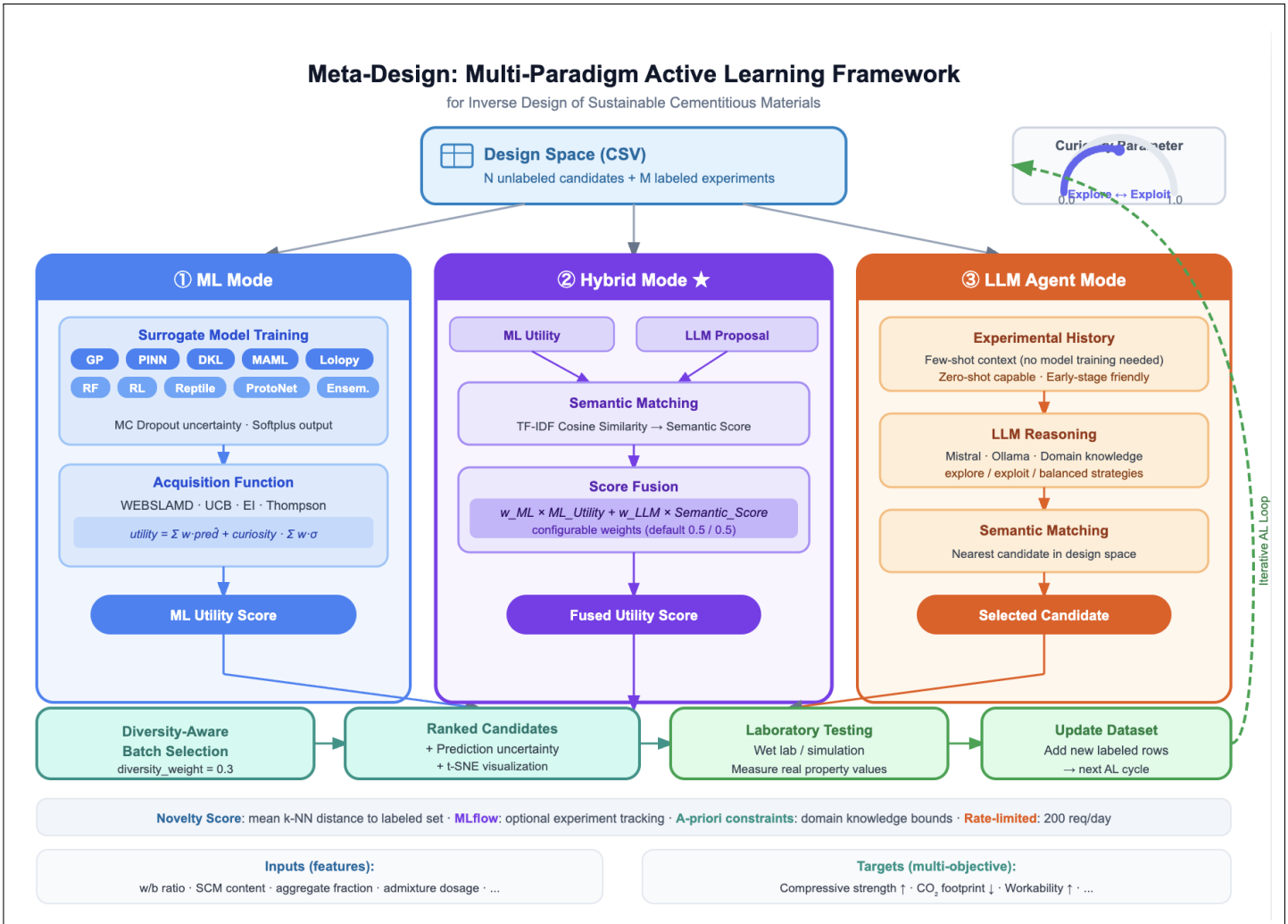

Fig. 1 Overview of the MetaDesign multi-paradigm active learning framework. The design space (CSV) feeds into three operating modes: (1) ML Mode, which trains a surrogate model (GP, PINN, DKL, MAML, RF, RL, Reptile, ProtoNet, Ensemble) and scores candidates using acquisition functions (WEBSLAMD, UCB, EI, Thompson Sampling); (2) Hybrid Mode, which combines ML utility with LLM-based semantic scoring via cosine similarity ($u_i^{\text{hybrid}} = w_{\text{ML}} \cdot u_i^{\text{ML}} + w_{\text{LLM}} \cdot s_i^{\text{LLM}}$); and (3) LLM Agent Mode, which selects candidates using LLM reasoning without surrogate training. All modes feed into a diversity-aware batch selection step, producing ranked candidates with uncertainty estimates and t-SNE visualization. Experimental results are used to update the dataset for the next active learning cycle. The curiosity parameter $\alpha \in [0, 1]$ controls the exploration–exploitation balance.

of true values fall within $\pm 1\hat{\sigma}$. Lolopy trains a separate model per target and requires a minimum of eight labeled samples.

3.1.3 Physics-Informed Neural Network.

The PINN[8] augments a three-layer MLP (128 neurons, ReLU, dropout 0.3) with a physics residual loss term weighted by $\lambda = 0.1$:

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{data}} + \lambda \cdot \mathscr{L}_{\text{physics}} \tag{2}$$

Universal constraints (non-negativity, smoothness, anti-collapse) are applied to all datasets. Domain-specific constraints are applied automatically from feature names: concrete datasets with `water` and `cement` columns receive Abrams' law constraints, while metal, polymer, and battery datasets receive domain-appropriate bounds. MC Dropout (30 passes) provides uncertainty.

3.1.4 Deep Kernel Learning.

DKL combines a two-layer neural network feature extractor (64 neurons, ReLU, dropout 0.2) with a GP operating in the learned feature space using a composite kernel (ConstantKernel × RBF + WhiteKernel). The network is pre-trained via a proxy MSE loss before features are passed to the GP, combining deep representation learning with principled Bayesian uncertainty.

3.1.5 Meta-learning models.

MAML[9] uses first-order MAML with a residual MLP (BatchNorm, dropout 0.3, Softplus output), AdamW with cosine annealing, and early stopping (patience = 20). Datasets with fewer than eight rows are augmented by tiling with Gaussian noise. A `MAMLMultiTargetWrapper` trains one model per target to prevent output collapse. Reptile[10] uses a three-layer MLP with LayerNorm, skip connections, and a decaying step size ($0.1 \rightarrow 0.01$).

Table 1 Surrogate model portfolio in MetaDesign. All models implement a common `predict_with_uncertainty` interface. MC = Monte Carlo; LOO-CV = leave-one-out cross-validation

Model	Method	Architecture	Uncertainty	Min. samples
GP	Bayesian	Matérn + WhiteKernel, one GP per target	Closed-form posterior σ	5
Random Forest (RF)	Ensemble	100 trees + StandardScaler	Inter-tree variance	5
Lolopy RF	Ensemble	Jackknife-after-bootstrap	Calibrated LOO-CV intervals	8
PINN	Physics-informed NN	3-layer MLP + physics residual loss	MC Dropout (30 passes)	5
DKL	Deep kernel learning	NN extractor + GP (RBF + WhiteKernel)	GP posterior σ	5
MAML	Meta-learning	MLP + BatchNorm + skip + FO-MAML	MC Dropout (50 passes)	3
Reptile	Meta-learning	3-layer MLP + LayerNorm + skip	MC Dropout (50 passes)	3
ProtoNet	Few-shot learning	Encoder + prototype comparison	MC Dropout	10
RL (PPO)	Reinforcement learning	Actor-critic, shared 128-neuron backbone	Policy distribution	5
Ensemble	Model combination	PINN + RF, uncertainty-weighted fusion	Weighted combination	5

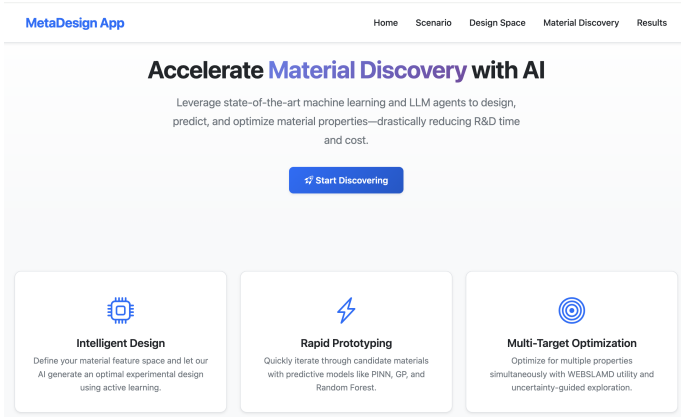

Fig. 2 MetaDesign landing page showing the main navigation bar and module overview, including the Design Space Builder, AL Dashboard, Results Viewer, and Scenario Management modules.

ProtoNet[11] embeds samples in a latent space and predicts targets via distance to labeled cluster centroids (prototypes), providing a naturally few-shot inference mechanism.

3.1.6 Reinforcement Learning.

The RL model implements PPO[13] with an actor-critic architecture. The state vector per candidate is:

$$\mathbf{s}_i = [\mathbf{x}_i \mid \bar{\mathbf{x}}_{\text{train}} \mid n_{\text{train}} \mid d_i]$$

where $\mathbf{x}_i$ are candidate features, $\bar{\mathbf{x}}_{\text{train}}$ is the mean of labeled samples, n_{train} is the number of labeled samples, and d_i is the Euclidean distance from the training centroid. The dual reward signal is:

$$r = 0.5 \cdot \Delta\text{RMSE} + 0.5 \cdot \Delta y_{\text{best}} \tag{3}$$

The RL model is checkpointed across sessions, enabling continuous policy learning over extended campaigns. At evaluation, RL scores are fused with WEBSLAMD utility: $u_i^{\text{final}} = (1 - w_{\text{RL}}) \cdot u_i + w_{\text{RL}} \cdot \pi_i$, with default $w_{\text{RL}} = 0.3$.

3.1.7 Ensemble.

The Ensemble model combines PINN and RF predictions via uncertainty-weighted fusion, assigning higher weight to the model that is more confident in a given candidate region.

3.2 Active learning modes

MetaDesign provides three operating modes, summarized in Table 2.

Table 2 Active learning modes in MetaDesign

Mode	Engine	Best used when
ML Mode	Surrogate model + acquisition function	Large labeled dataset; well-understood system
LLM Agent	LLM reasoning (Ollama / Mistral)	Very small dataset (<10 labels); strong domain priors
Hybrid	ML + LLM fusion (TF-IDF cosine)	Moderate dataset; best of both worlds

3.2.1 ML Mode.

The surrogate model is trained on labeled data, predictions and uncertainties are computed for all unlabeled candidates, and candidates are ranked by WEBSLAMD utility (Eq. 1). This mode replicates and extends the SLAMD/WEBSLAMD approach[1] with the expanded model portfolio.

3.2.2 LLM Agent Mode.

No surrogate model training is performed. The experimental history, optimization objectives, parameter space description, and a strategy instruction (*explore*, *exploit*, or *balanced*) are passed to an LLM (locally hosted Ollama or Mistral Cloud API, temperature = 0 for reproducibility). The LLM proposes optimal parameters as a JSON object, and a TF-IDF semantic matcher finds the nearest candidate via cosine similarity. The LLM also predicts expected target values for the selected candidate, making this mode viable with as few as zero labeled samples.

3.2.3 Hybrid Mode.

Hybrid Mode is the most sophisticated operating mode, fusing statistical and knowledge-driven signals. The ML surrogate produces per-candidate utility scores u_i^{ML}. Concurrently, the LLM generates an ideal experiment description in natural language. TF-IDF vectorization converts all candidates to text representations, and cosine similarity with the LLM proposal yields per-candidate semantic scores s_i^{LLM}. The fused utility is:

$$u_i^{\text{hybrid}} = w_{\text{ML}} \cdot u_i^{\text{ML}} + w_{\text{LLM}} \cdot s_i^{\text{LLM}} \tag{4}$$

with default weights $w_{\text{ML}} = w_{\text{LLM}} = 0.5$ (user-configurable). A diversity-aware batch selection step (diversity weight = 0.3) sub-

sequently selects a batch of candidates covering distinct feature-space regions to reduce experimental redundancy.

3.3 Acquisition functions

Four acquisition functions are available as hot-swappable components implementing a common abstract base class (Table 3). The WEBSLAMD function (Eq. 1) is the default and is directly compatible with the SLAMD framework.[1] UCB and EI are classical Bayesian optimization functions;[5] their exploration parameters scale with the curiosity setting α. Thompson Sampling provides stochastic exploration.

Table 3 Acquisition functions in MetaDesign

Function	Formula	Exploration bias
WEBSLAMD	Eq. 1	Curiosity $\alpha \in [0,1]$
UCB	$\mu + \beta\sigma$	β scales with α
EI	$(\mu - f^*)\Phi(Z) + \sigma\phi(Z)$	ξ scales with α
Thompson	Sample $\sim \mathcal{N}(\mu, \sigma^2)$	Stochastic

3.4 Design Space Builder

The Design Space Builder (Fig. 3) enables construction of structured candidate pools without requiring existing laboratory data. Users define each feature as continuous (minimum, maximum, step size or number of points) or discrete/categorical (comma-separated values). The full Cartesian product is generated via `itertools.product` and exported as a CSV with empty target columns ready for the AL pipeline. In-place browser-based cell editing allows manual annotation of known values, and the system warns when the combination count exceeds 100,000 rows.

3.5 Results table and visualizations

After each AL iteration, MetaDesign renders a ranked results table with a *Selected for Testing* flag on the highest-utility candidates, alongside a comprehensive visualization dashboard (Fig. 4). The t-SNE projection of the full design space — shown in Fig. 4 — color-codes all candidates by predicted utility, with labeled training points displayed as crosses and unlabeled candidates as circles, enabling practitioners to identify promising and unexplored regions simultaneously. Additional panels include: an optimization history curve (best observed target value *vs.* labeled sample count); per-candidate uncertainty distributions; a 3D utility surface over the first two t-SNE dimensions; Euclidean distance plots between consecutive selections; and feature importance via absolute Pearson correlation with the primary target. A leave-one-out k-nearest-neighbour prediction-vs-actual plot assesses model quality without requiring a held-out test set.

The exploration trajectory visualization (Fig. 5) complements the t-SNE scatter plot by explicitly tracing the *path* of selected candidates through feature space across iterations, color-coded by the AL mode used at each step (ML Mode, LLM Agent, or Hybrid). The total cumulative distance traversed in t-SNE space is

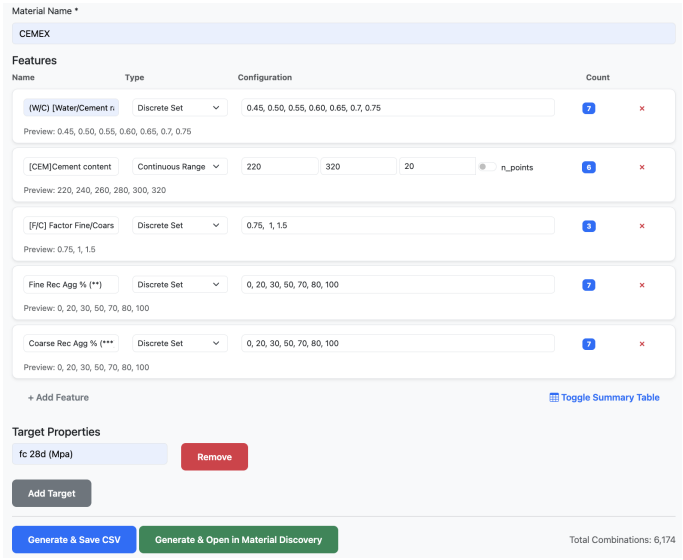

Fig. 3 MetaDesign Design Space Builder. Left: feature definition panel for specifying continuous ranges (min, max, step) or discrete categorical values. Right: generated design space table with in-place editing, row count display, and download/delete controls.

reported in the plot header, quantifying the breadth of exploration achieved over the campaign. This allows practitioners to diagnose whether the engine is exploiting a narrow region or exploring the full design space.

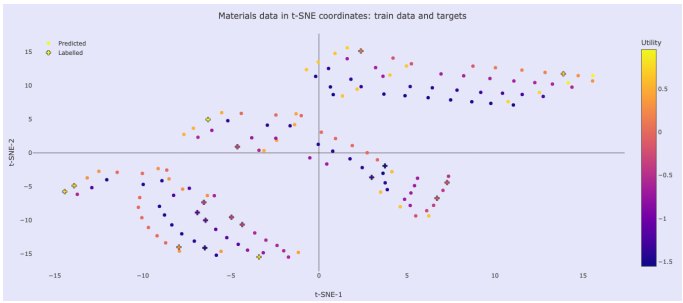

Fig. 4 MetaDesign discovery dashboard. The t-SNE design space projection colors all candidates by predicted utility (color bar, right), with labeled training samples shown as crosses (+) and unlabeled candidates as filled circles. High-utility regions (warm colors) are clearly separated from low-utility regions (cool colors), guiding the practitioner toward the most promising formulations. The dashboard additionally provides an optimization history curve, uncertainty distribution, 3D utility surface, and feature importance panel (not shown).

3.6 Multi-objective optimization

MetaDesign supports multi-objective optimization through three scalarization strategies: weighted sum, ParEGO (random Dirichlet-sampled weights per call to promote Pareto front exploration), and a Pareto-based strategy where each objective is scored independently. Both maximization and minimization targets are supported; predictions for minimization targets are

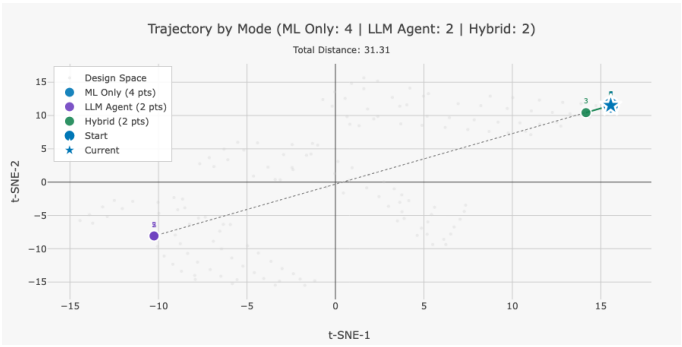

Fig. 5 Exploration trajectory plot in MetaDesign, showing the path of selected candidates through t-SNE feature space across active learning iterations. Points are color-coded by operating mode: ML Mode (blue, 4 selections), LLM Agent Mode (purple, 2 selections), and Hybrid Mode (green, 2 selections). The dashed line traces the sequential selection path; the total cumulative distance (31.31 t-SNE units in this example) quantifies the breadth of design space coverage achieved over the campaign. The star marker denotes the most recently selected candidate.

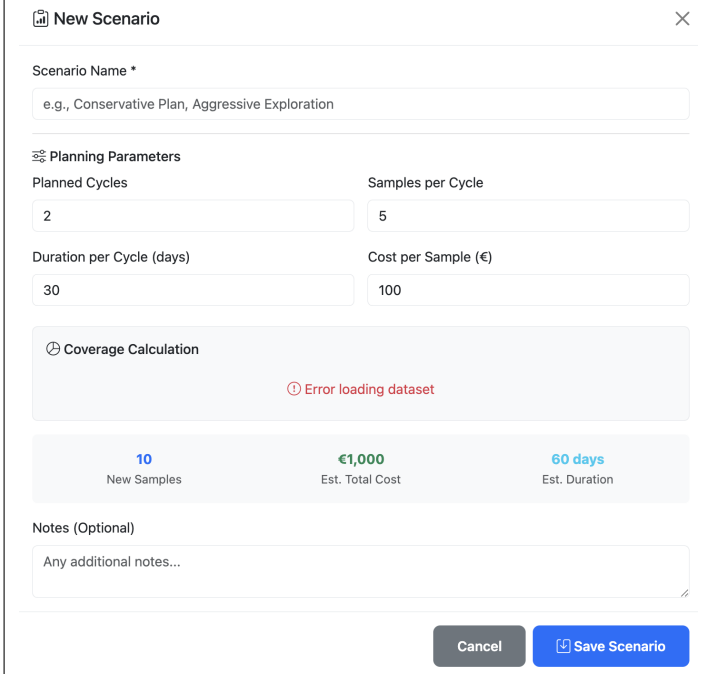

Fig. 6 MetaDesign scenario creation dialog. Users define planning parameters (number of cycles, samples per cycle, cycle duration, and cost per sample). The system computes real-time coverage estimates, including total samples, total cost, campaign duration, and design space coverage, enabling pre-experimental budgeting.

negated before acquisition scoring.

3.7 Feature preprocessing

Automated preprocessing removes constant columns (zero variance), columns with pairwise Pearson correlation exceeding 0.99, and handles missing values by imputing numeric NaN entries with column means. Infinite values are replaced with NaN prior to imputation. A validation report identifies all dropped columns and preprocessing warnings.

3.8 Project and scenario management

MetaDesign includes a SQLite-backed project and scenario management system that organizes AL campaigns at two hierarchical levels. A *Project* is a top-level experimental container; each project can contain multiple *Scenarios* that define distinct experimental plans. As shown in Fig. 6, each scenario specifies: the number of planned AL cycles, samples per cycle, duration per cycle (days), and cost per sample (). A real-time coverage calculation panel automatically computes the total number of new samples, estimated total cost, estimated campaign duration, and the resulting estimated coverage of the design space as a percentage. For example, a scenario with 2 planned cycles of 5 samples each, at 100 per sample, over a design space of 12,348 candidates, yields 10 new samples, 1,000 estimated cost, 60 days estimated duration, and 0.1% estimated design space coverage. Progress is tracked per project and exposed via real-time API endpoints at `/api/scenarios/projects/<id>/progress`.

4 Workflow

The MetaDesign workflow follows six successive steps, also illustrated in Fig. 1. Before beginning, practitioners may use the scenario management system (Fig. 6) to pre-plan the campaign budget, cycle count, and expected design space coverage.

1. **Define the design space.** Use the Design Space Builder to generate a Cartesian product of features, or upload an existing CSV/Excel file with optional partial labels.

2. **Label an initial set.** Provide 5–15 experimental results as labeled rows. These may be imported from existing records or obtained from a random initial batch.

3. **Configure the run.** Select the surrogate model, AL mode (ML, LLM Agent, or Hybrid), target columns and optimization directions, importance weights, curiosity α, batch size, and acquisition function.

4. **Run the AL engine.** The engine trains the surrogate on labeled data, computes WEBSLAMD utility for all unlabeled candidates, and produces a ranked recommendation list with *Selected for Testing* flags.

5. **Validate in the laboratory.** Test the recommended candidates and record results in the dataset.

6. **Repeat.** The updated dataset is used to retrain the surrogate; the cycle continues until the target performance criteria are achieved or the budget is exhausted.

5 Discussion

MetaDesign extends the SLAMD framework [1] in several important directions. Expanding from two surrogate models to ten covers a broad spectrum of dataset sizes, complexity levels, and prior

knowledge availability. The PINN model is particularly noteworthy for building materials: physical relationships such as Abrams' law for concrete strength are well-established and can meaningfully regularize model behaviour in data-sparse regimes. The meta-learning models (MAML, Reptile, ProtoNet) address a common industrial challenge: new raw material sources or binding systems may have very little historical data yet share structural similarities with previously studied systems, enabling rapid adaptation.

The LLM Agent and Hybrid modes represent a conceptually novel contribution to the materials discovery software landscape. Domain knowledge accumulated in scientific literature — covering, for instance, the expected effect of slag content on reactivity or the role of the water-to-binder ratio on workability — is encoded implicitly in LLM pretraining. Making this knowledge accessible within the AL loop, without requiring explicit expert-system programming, has the potential to substantially improve early-stage campaign efficiency. This is especially relevant in the context of circular economy materials, where waste-derived binders are compositionally variable and historical databases are sparse.[3]

The RL model offers a distinct advantage over all other surrogate approaches: rather than predicting properties and scoring candidates with a fixed function, it learns a selection policy that compounds improvements over a campaign's lifetime. While RL requires more iterations to demonstrate its advantage, in long-running experimental programs it has the potential to discover system-specific exploration strategies that outperform fixed acquisition functions.

MetaDesign inherits and advances the open-science philosophy of SLAMD.[1] The open-source codebase supports community contributions and adaptation to material domains beyond concrete, including metals, polymers, and battery materials, for which domain-specific PINN constraints have already been implemented. The exploration trajectory visualization (Fig. 5) provides an additional audit tool: by tracing the path of selected candidates through t-SNE feature space and color-coding selections by operating mode, it enables post-hoc analysis of whether the campaign explored broadly or converged prematurely.

Conclusions

We have introduced MetaDesign, a multi-paradigm active learning framework for the inverse design of sustainable cementitious and other materials. Built upon the SLAMD framework,[1] MetaDesign extends it with ten surrogate model architectures, three operating modes including a novel LLM–ML hybrid, four acquisition functions, multi-objective optimization, an interactive design space builder, comprehensive visualizations, and project management. The WEBSLAMD utility function (Eq. 1) provides a common scoring backbone across all models and modes, ensuring backward compatibility with existing SLAMD workflows.

By integrating statistical, physics-informed, meta-learning, reinforcement learning, and LLM-based reasoning into a unified web application, MetaDesign substantially lowers the barrier to data-driven materials discovery for laboratory practitioners across a wide range of expertise levels.

Future work will benchmark the surrogate portfolio on standardized building materials datasets, evaluate the Hybrid Mode against pure ML baselines across campaign sizes, and extend domain-specific PINN constraints to geopolymers and calcium sulfoaluminate cements.

Author contributions

G.A.Z.: conceptualization, software development, writing – original draft. C.V.: conceptualization, methodology, supervision, writing – review & editing. B.M.T.: methodology, writing – review & editing. R.F.: resources, validation, writing – review & editing. D.S.: supervision, funding acquisition. S.K.: supervision, funding acquisition, writing – review & editing.

Conflicts of interest

There are no conflicts to declare.

Data availability

MetaDesign is open-source and available at `https://github.com/BAMresearch/MetaDesign`. All data used in the examples presented in this work are available within the repository.

Acknowledgements

The authors gratefully acknowledge support from the European Union's Horizon Europe research and innovation programme under grant agreement No. 101056773 (Reincarnate project).

Notes and references

For the reference section, the style file `rsc.bst` can be used to generate the correct reference style.§

1. Citations should appear here in the format A. Name, B. Name and C. Name, *Journal Title*, 2000, **35**, 3523;

2. A. Name, B. Name and C. Name, *Journal Title, 2000*, **35**, 3523.

...

We encourage the citation of primary research over review articles, where appropriate, in order to give credit to those who first reported a finding. Find out more about our commitments to the principles of San Francisco Declaration on Research Assessment (DORA).

1 C. Völker, B. Moreno Torres, G. A. Zia, R. Firdous, T. Rug, F. Böhmer, D. Stephan and S. Kruschwitz, *NanoWorld J.*, 2023, **9**, S180–S187.

2 C. Völker, R. Firdous, D. Stephan and S. Kruschwitz, *J. Mater. Sci.*, 2021, **56**, 15859–15881.

3 C. Völker, B. Moreno Torres, T. Rug, R. Firdous, G. A. Zia, D. Stephan and S. Kruschwitz, *J. Clean. Prod.*, 2023, **418**, 138221.

4 A. Zunger, *Nat. Rev. Chem.*, 2018, **2**, 0121.

5 T. Lookman, P. V. Balachandran, D. Xue and R. Yuan, *npj Comput. Mater.*, 2019, **5**, 21.

6 B. Rohr, H. S. Stein, D. Guevarra, Y. Wang, J. A. Haber, M. Aykol, S. K. Suram and J. M. Gregoire, *Chem. Sci.*, 2020, **11**, 2696–2706.

7 J. Ling, M. Hutchinson, E. Antono, S. Paradiso and B. Meredig, *Integr. Mater. Manuf. Innov.*, 2017, **6**, 207–217.

8 M. Raissi, P. Perdikaris and G. E. Karniadakis, *J. Comput. Phys.*, 2019, **378**, 686–707.

9 C. Finn, P. Abbeel and S. Levine, Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1126–1135.

10 A. Nichol, J. Achiam and J. Schulman, *arXiv preprint arXiv:1803.02999*, 2018.

11 J. Snell, K. Swersky and R. S. Zemel, Adv. Neural Inf. Process. Syst., 2017, pp. 4077–4087.

12 D. A. Boiko, R. MacKnight, B. Kline and G. Gomes, *Nature*, 2023, **624**, 570–578.

13 R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA, 2nd edn, 2018.

14 Z. Zhou, S. Kearnes, L. Li, R. N. Zare and P. Riley, *Sci. Rep.*, 2019, **9**, 10752.

15 F. Pedregosa *et al.*, *scikit-learn: Gaussian Process Regressor*, `https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.GaussianProcessRegressor.html`, 2024, Accessed 2025.

16 Citrine Informatics, *lolopy: Python wrapper for LoLo*, `https://pypi.org/project/lolopy/`, 2023, Accessed 2025.